

Assignment Title

COURSE NAME

Big Data Analytics

Author:

Simone Sbreglia

21 aprile 2025

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



Indice

Abstract	2
1 MongoDB	2
1.1 Introduzione	2
1.2 Modalità di utilizzo	2
1.3 Operazioni CRUD	3
1.3.1 Insert	4
1.3.2 Delete	4
1.3.3 Update	4
1.3.4 Read	5
1.4 Aggregazione	5
2 Sperimentazione	6
References	8
A Additional graphs and data	9

Abstract

In questo report si esporrà il funzionamento e le caratteristiche di MongoDB, un database NoSQL. In particolare verranno presentate le funzionalità di questo database, il modo in cui si possono memorizzare i dati e come effettuare le principali operazioni CRUD (Create, Read, Update, Delete). In questo specifico caso di utilizzo MongoDB è stato utilizzato per memorizzare i dati di un negozio musicale, riadattando la struttura nativa relazionale del database in una struttura NoSQL più adatta a MongoDB. Tutte le operazioni sono state effettuate tramite python, utilizzando la libreria pymongo. Infine è stata sviluppata una interfaccia grafica in streamlit per interrogare in modo dinamico il database, sia per la parte musicale che per la parte amministrativa e di vendite. Il codice sorgente è disponibile al repository GitHub disponibile al link fornito.

1 MongoDB

1.1 Introduzione

MongoDB è un database NoSQL, ossia un database non relazionale che archivia i dati in modo flessibile. MongoDB offre un servizio su Cloud, MongoDB Atlas e una versione MongoDB Community, installabile sulla propria macchina in locale. Per gli scopi di questo progetto è stato installato MongoDB Community in locale. MongoDB è pensato per essere scalato orizzontalmente, distribuendo i dati su più macchine e con varie repliche, per garantire alta disponibilità e tolleranza ai guasti.

Un record in MongoDB è un documento, ossia una struttura dati composta da coppie di chiavi a valori; questi valori potrebbero essere a loro volta documenti, oppure vettori o valori. La struttura di un documento è simile a quella di un oggetto JSON, ossia un oggetto JavaScript che rappresenta una struttura dati composta da coppie di chiavi a valori. In particolare MongoDB salva i documenti in formato BSON, ossia una rappresentazione binaria dei file JSON. Un esempio di documento è il seguente:

Listing 1: Esempio di JSON

```
1 {
2   "Nome": "Paolo",
3   "Professione": "Studente",
4   "TitoliDiStudio": ["Diploma", "Laurea Triennale"],
5   "Indirizzo": {
6     "Via": "Campi"
7     "NumeroCivico": "213/b",
8     "Cap": "41125",
9     "Citta": "Modena"
10  },
11  "Anni": 24,
12 }
```

L'utilizzo di documenti come record per il salvataggio di dati ha diversi vantaggi, infatti documenti di questo tipo corrispondono al tipo nativo di dati in molti linguaggi di programmazione. Inoltre la possibilità di annidare documenti e di inserire array come valori permette di avere strutture di dati che diminuiscono le operazioni di join. MongoDB memorizza i documenti in *collezioni*, ossia l'equivalente delle tabelle in database relazionali. Inoltre MongoDB supporta anche:

- **Viste di sola lettura:** viste virtuali basate su una query che mostrano una finestra sui dati di una collezione esistente, aggiornata automaticamente ogni volta che viene consultata la finestra
- **Viste materializzate su richiesta:** una tabella o una collezione che memorizza i risultati di una query per migliorare le prestazioni, non vengono aggiornate automaticamente ma solo manualmente o tramite script

1.2 Modalità di utilizzo

MongoDB può essere utilizzato in diversi modi, tra cui:

- MongoDB Shell
- MongoDB Compass
- MongoDB Atlas

- Driver MongoDB

MongoDB Shell è un'interfaccia a riga di comando che permette di interagire con il database, eseguire query e comandi di amministrazione. La shell risulta utile per eseguire operazioni rapide e testare comandi, ma non è adatta all'uso in produzione.

MongoDB Compass è un'interfaccia grafica che permette di esplorare i dati, eseguire query e visualizzare i risultati, fornisce uno strumento utile per analizzare i dati e per eseguire operazioni di amministrazione, ma non è adatta per l'uso in produzione.

MongoDB Atlas è un servizio di database as a service (DBaaS) che permette di creare e gestire cluster MongoDB in cloud. Atlas offre funzionalità avanzate come il backup automatico, la scalabilità automatica e la sicurezza integrata previo pagamento, ma mette a disposizione anche un piano gratuito con una capacità massima di 512 MB e alcune limitazioni per backup e replica.

Driver MongoDB sono librerie che permettono di interagire con MongoDB da diversi linguaggi di programmazione, come Python, Java, Node.js e altri. I driver offrono un'interfaccia per eseguire operazioni CRUD e per gestire le connessioni al database. Per questo progetto è stato utilizzato il driver **pymongo** per Python. Utilizzare un driver significa potersi appoggiare a un linguaggio di programmazione per costruire query ed effettuare operazioni in modo dinamico e automatico.

1.3 Operazioni CRUD

Le operazioni CRUD (Create, Read, Update, Delete) sono le operazioni fondamentali per interagire con un database. In MongoDB queste operazioni vengono eseguite tramite metodi specifici. Di seguito sono riportati alcuni esempi di operazioni CRUD utilizzando MongoDB Shell. Supponiamo di avere in locale MongoDB Compass e MongoDB Shell installati, allora le istruzioni a seguire permettono di creare un database *Scuola* e due collezioni, *Studenti* e *Professori*.

Listing 2: Esempio di operazioni CRUD in MongoDB Shell

```
1 use Scuola
2 db.createCollection(
3     "Studenti", {
4         validator: {
5             $jsonSchema: {
6                 bsonType: "object",
7                 title: "Validazione dell'oggetto Studente",
8                 required: ["Id", "Nome", "Cognome"],
9                 properties: {
10                     Id: { bsonType: "int" },
11                     Nome: { bsonType: "string" },
12                     Cognome: { bsonType: "string" },
13                     DataDiNascita: { bsonType: "date" }
14                 }
15             }
16         }
17     } )
18 db.Studenti.createIndex({ "Id": 1 }, { unique: true })
19 db.createCollection(
20     "Professori", {
21         validator: {
22             $jsonSchema: {
23                 bsonType: "object",
24                 title: "Validazione dell'oggetto Studente",
25                 required: ["Id", "Nome", "Cognome"],
26                 properties: {
27                     Id: { bsonType: "int" },
28                     Nome: { bsonType: "string" },
29                     Cognome: { bsonType: "string" },
30                     DataDiNascita: { bsonType: "date" }
31                 }
32             }
33         }
34     } )
35 db.Professori.createIndex({ "Id": 1 }, { unique: true })
```

Le operazioni CRUD mostrate creano un database Scuola, due collezioni, Studenti e Professori, e definiscono uno schema per la validazione dei documenti: è richiesto che siano inseriti almeno un Id, un Nome e un Cognome. Inoltre viene richiesto, tramite la creazione un indice, che gli Id siano univoci nelle rispettive collezioni. Se durante una fase di insert non vengono soddisfatti questi requisiti, MongoDB non permette di inserire il documento e restituisce un errore.

1.3.1 Insert

MongoDB permette di inserire documenti tramite dei metodi, *insert_one* e *insert_many*, che nello specifico inseriscono uno o più documenti in una collezione. Di seguito sono riportati alcuni esempi di inserimento di documenti.

Listing 3: Esempio di insert in MongoDB

```
1 db.Studenti.insertOne({
2     Id: 2,
3     Nome: "Luca",
4     Cognome: "Rossi",
5     DataDiNascita: ISODate("2000-05-15T00:00:00Z")
6 })
7 db.Studenti.insertMany([
8     {
9         Id: 2,
10        Nome: "Stefano",
11        Cognome: "Verdi",
12        DataDiNascita: ISODate("2000-05-15T00:00:00Z")
13    },
14    {
15        Id: 3,
16        Nome: "Anna",
17        Cognome: "Bianchi",
18        DataDiNascita: ISODate("1999-11-20T00:00:00Z")
19    },
20    {
21        Id: 4,
22        Nome: "Marco",
23        Cognome: "Galli",
24        DataDiNascita: ISODate("2001-01-10T00:00:00Z")
25    }
26 ])
```

In modo analogo si possono inserire dei documenti nella collezione Professori.

1.3.2 Delete

MongoDB permette di eliminare documenti tramite i metodi *delete_one* e *delete_many*, che eliminano uno o più documenti in una collezione. Di seguito sono riportati alcuni esempi di eliminazione di documenti.

Listing 4: Esempio di delete in MongoDB

```
1 db.Studenti.deleteOne({ Id: 2 })
2 db.Studenti.deleteMany({ Nome: "Marco" })
3 db.Studenti.deleteMany({ Nome: { $in: ["Marco", "Luca"] } })
4 db.Studenti.deleteMany({ Nome: { $nin: ["Marco", "Luca"] } })
```

Il primo comando elimina il documento con Id 2, il secondo elimina tutti i documenti in cui il campo "Nome" è "Marco", il terzo elimina tutti i documenti in cui il campo "Nome" è "Marco" o "Luca", mentre il quarto elimina tutti i documenti in cui il campo "Nome" non è "Marco" o "Luca".

1.3.3 Update

MongoDB permette di aggiornare documenti tramite i metodi *update_one* e *update_many*, che aggiornano uno o più documenti in una collezione. Durante gli update vengono spesso usate pipeline di aggregazione, che permettono di concatenare più azioni una dopo l'altra. In seguito verranno approfondite le pipeline di aggregazione, per il momento sono riportati alcuni esempi di aggiornamento di documenti.

Listing 5: Esempio di update in MongoDB

```

1  db.Studenti.updateOne(
2      { Id: 2 },
3      { $set: { Nome: "Marco" } }
4  )
5  db.Studenti.updateMany(
6      { Nome: "Marco" },
7      { $set: { Nome: "Luca" } }
8  )
9  db.Studenti.updateMany(
10     { Corso: { $exists: false } },
11     { $set: { Corso: "Matematica" } }
12 )

```

Il primo comando aggiorna il documento con Id 2, cambiando il campo Nome in Marco, il secondo cambia il campo Nome in Luca per tutti i documenti in cui il campo Nome è Marco, mentre il terzo aggiorna tutti i documenti in cui il campo Corso non esiste, aggiungendo il campo Corso con valore Matematica.

1.3.4 Read

MongoDB permette di leggere documenti tramite i metodi *findOne*, *find* che restituiscono uno o più documenti in una collezione. Entrambi i metodi prendono in input un filtro che permette di specificare quali documenti si vogliono leggere, espressi in formato JSON, e un parametro di proiezione che permette di specificare quali campi di vogliono restituire e con quale nome, sempre in formato JSON. Di seguito sono riportati alcuni esempi di lettura di documenti.

Listing 6: Esempio di read in MongoDB

```

1  db.Studenti.find()
2  db.Studenti.find({ Id: 2 })
3  db.Studenti.find({ Corso: "Matematica" })
4  db.Studenti.find(
5      { Corso: "Matematica" },
6      { Nome: 1, Cognome: 1, _id: 0 }
7  )

```

Il primo comando restituisce tutti i documenti nella collezione Studenti, il secondo restituisce il documento con Id 2, il terzo restituisce tutti i documenti in cui il campo Corso è Matematica, mentre il quarto restituisce solo i campi Nome e Cognome dei documenti in cui il campo Corso è Matematica, escludendo il campo `_id`.

1.4 Aggregazione

MongoDB mette a disposizione un potente framework di aggregazione che permette di eseguire operazioni complesse sui dati, come filtri, ordinamenti, raggruppamenti, join, somme e molto altro. Le operazioni di aggregazione non sono altro che delle pipeline che contengono un operatore dopo l'altro, ognuno dei quali esegue un'operazione sui dati e restituisce il risultato al successivo operatore. In Tabella 1, sono riportati i principali operatori di aggregazione disponibili in MongoDB. Per visualizzare la lista completa degli operatori di aggregazione è possibile consultare la documentazione ufficiale di MongoDB, vedi [2].

Operatore	Descrizione
\$match	Filtra i documenti secondo una condizione, simile a una WHERE in SQL.
\$group	Raggruppa i documenti in base a uno o più campi e calcola valori aggregati.
\$sort	Ordina i documenti secondo uno o più campi, in ordine crescente o decrescente.
\$project	Seleziona e ristruttura i campi del risultato, può anche creare campi nuovi.
\$limit	Limita il numero di documenti nel risultato finale.
\$skip	Salta un certo numero di documenti dall'inizio del risultato.
\$count	Conta il numero di documenti e restituisce il totale in un campo chiamato count.
\$lookup	Effettua una "join" tra collezioni, simile a una JOIN in SQL.
\$unwind	Divide un array in più documenti singoli, uno per ogni elemento dell'array.
\$addFields	Aggiunge nuovi campi o modifica quelli esistenti nei documenti.
\$replaceRoot	Sostituisce il documento intero con un altro oggetto specificato.
\$merge	Scriva il risultato dell'aggregazione in una nuova collezione o aggiorna una esistente.

Tabella 1: Operatori di aggregazione più comuni in MongoDB

La concatenazione di pipeline di aggregazione permette di eseguire operazioni complesse direttamente sul database, sfruttando tutte le ottimizzazioni e le potenzialità di MongoDB. Per alcuni esempi, vedere la Sezione [2](#).

2 Sperimentazione

Lo scopo del progetto è quello di sviluppare competenze legate a MongoDB, capire come funziona, come gestire i dati e come sfruttare il database da altri linguaggi di programmazione, come ad esempio Python.

A tale scopo è stato scelto un dataset, chiamato Chinook Database [\[1\]](#). Il database originale è un database relazionale, composto da 11 tabelle, che raccolgono informazioni su tracce musicali, artisti, album e alcune playlist, insieme alle informazioni sulle vendite, sui clienti e sugli impiegati del negozio musicale. In Figura [1](#) è riportato lo schema del database originale, composto da 11 tabelle. Il database è stato convertito in un formato NoSQL,

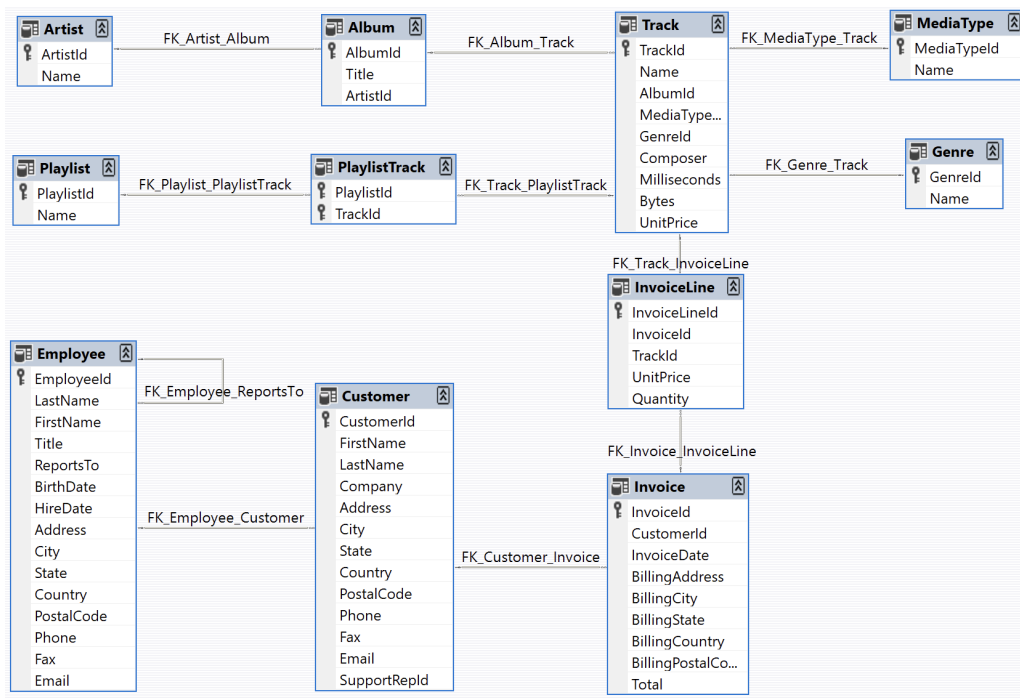


Figura 1: Schema del database Chinook

Riferimenti bibliografici

[1] Kaggle (2025). Chinookdataset. Accessed: 2025-04-21.

[2] MongoDB, Inc. (2025). Mongoddb documentation. Accessed: 2025-04-21.

A Additional graphs and data